Universidad Veracruzana

Ingeniería en instrumentación electrónica

Programación Orientada a objetos

José Alfonso Domínguez Chaves

Equipo 1:

Los Compilados

Hurtado Vasconcelos Rafael Alejandro

Romero Hernández Luis Horacio

Vázquez Espinosa Santiago Maximiliano

Landa Cervantes Juan Giovanni

14 de octubre de 2026 Xalapa, ver

1 - Introducción

Actualmente, el Internet de las Cosas (IoT) tiene una gran importancia en el desarrollo de sistemas tecnológicos, debido a que permite conectar diferentes dispositivos para recopilar información, supervisar variables y realizar acciones de manera automatizada. Esta tecnología se encuentra presente en aplicaciones industriales, sistemas de monitoreo ambiental, automatización de viviendas, agricultura inteligente y procesos de control.

En esta práctica se propone desarrollar una simulación de una red IoT utilizando Python, aplicando tres principios importantes de la programación orientada a objetos: herencia, encapsulamiento y polimorfismo.

El sistema estará compuesto por una clase principal llamada NodoIoT, que contendrá las características generales de los dispositivos, y dos clases derivadas: NodoSensor y NodoActuador.

A partir de estas clases se crearán tres objetos que representarán un sensor de temperatura, un sensor de iluminación y un ventilador. Estos dispositivos se integrarán en una lista para simular una red en la que cada elemento pueda conectarse, mostrar información y ejecutar acciones específicas.

Además, se implementará un menú interactivo que permitirá al usuario consultar el estado de los dispositivos, visualizar su telemetría, modificar límites de alerta y controlar el funcionamiento del ventilador.

En una red IoT pueden existir diferentes dispositivos con características y funciones particulares. Algunos se encargan de recopilar información mediante sensores, mientras que otros ejecutan acciones sobre determinados procesos.

La práctica plantea resolver esta situación mediante el desarrollo de una jerarquía de clases en Python, utilizando herencia para compartir características, encapsulamiento para controlar los atributos y polimorfismo para gestionar diferentes tipos de dispositivos mediante una misma instrucción.

2 - Justificación

En el desarrollo de aplicaciones tecnológicas es común trabajar con varios dispositivos que requieren ser supervisados o controlados. Por ello, resulta necesario diseñar programas que puedan adaptarse a diferentes elementos sin tener que desarrollar nuevamente toda su estructura.

Mediante la herencia es posible definir características generales de un dispositivo y reutilizarlas en otros elementos con funciones más específicas.

El encapsulamiento permite proteger los atributos internos para evitar modificaciones incorrectas. Esto es especialmente útil cuando se trabaja con parámetros de funcionamiento, como los límites de alerta o los valores de potencia.

Por otra parte, el polimorfismo permite ejecutar una misma operación sobre distintos objetos, obteniendo respuestas particulares según el tipo de dispositivo.

El desarrollo de esta simulación también representa una oportunidad para comprender cómo se puede estructurar el software de sistemas de monitoreo y control.

Aunque en esta práctica no se realiza una conexión física con dispositivos electrónicos, la organización del programa puede servir como base para proyectos posteriores que utilicen microcontroladores, sensores reales, módulos de comunicación o interfaces de supervisión.

3 - Objetivos

Objetivos específicos

- Diseñar una clase principal denominada NodoIoT que contenga los atributos y métodos comunes de los dispositivos.
- Implementar las clases derivadas NodoSensor y NodoActuador mediante herencia.
- Utilizar la función super() para reutilizar el constructor de la clase padre.
- Aplicar el encapsulamiento mediante atributos privados y métodos de acceso controlado.
- Validar los valores numéricos ingresados para evitar configuraciones incorrectas.
- Sobrescribir el método enviar_telemetria() en las clases derivadas.
- Crear una lista mixta de dispositivos y recorrerla mediante un ciclo for.
- Implementar un menú interactivo para controlar las funciones de la red.
- Elaborar un diagrama UML que represente la estructura y relaciones entre las clases.
- Realizar pruebas de funcionamiento para verificar el comportamiento de los dispositivos.

4 - Materiales y herramientas

Para realizar la práctica se requiere una computadora con un entorno de programación compatible con Python.

Las herramientas consideradas son:

- Computadora personal.
- Lenguaje de programación Python 3.
- Visual Studio Code o cualquier editor compatible con Python.
- Terminal o consola para ejecutar el programa.
- Herramienta para elaborar el diagrama UML.
- Procesador de textos para redactar el reporte.

No se necesitan sensores, actuadores ni microcontroladores físicos, debido a que el comportamiento de la red se representará mediante software.

5 - Desarrollo de la práctica

Diseño del diagrama de clases UML

Antes de comenzar con la programación, se plantea un diagrama UML para representar la estructura general del sistema.

El diagrama estará formado por tres clases: NodoIoT, NodoSensor y NodoActuador.

La clase NodoIoT ocupará la parte superior por ser la superclase. Las otras dos clases se ubicarán debajo, conectadas mediante flechas de herencia con triángulo vacío apuntando hacia la clase padre.

En el diagrama se deberán incluir los atributos, tipos de datos, métodos y parámetros.

También se utilizarán los símbolos de visibilidad correspondientes:

- El símbolo + indica un atributo o método público.
- El símbolo - indica un atributo o método privado.

La representación permitirá identificar qué elementos se comparten entre los dispositivos y cuáles corresponden únicamente a una clase.

Evidencia 1. Insertar el diagrama UML del sistema IoT.

Programación de la clase padre NodoIoT

La primera etapa de programación consiste en crear la clase NodoIoT.

Esta clase funcionará como estructura principal del sistema y contendrá los atributos compartidos por todos los dispositivos.

Los atributos considerados son id_dispositivo, ip, ubicacion y conectado.

El identificador permitirá distinguir los diferentes dispositivos, mientras que la dirección IP representará su ubicación lógica dentro de una red. El atributo ubicacion indicará el lugar donde se encuentra instalado el nodo. Por último, conectado permitirá almacenar un valor booleano que indique si el dispositivo se encuentra conectado o desconectado dentro de la simulación. También se implementará el método conectar(), que modificará el estado del dispositivo para representar una conexión exitosa.

El método enviar_telemetria() se incluirá como comportamiento general que posteriormente será sobrescrito por las clases derivadas.

Evidencia 2. Insertar una captura del código correspondiente a la clase NodoIoT.

Programación de la clase NodoSensor

Una vez definida la clase padre, se desarrollará la clase NodoSensor.

Esta clase utilizará herencia para obtener las características generales de un nodo IoT.

Mediante super() se llamará al constructor de la superclase, permitiendo inicializar el identificador, la dirección IP, la ubicación y el estado de conexión. Posteriormente, se agregará el atributo privado __limite_alerta, que permitirá almacenar un valor de referencia para identificar condiciones que superen el límite establecido. El método setter será responsable de verificar que el valor recibido sea numérico, finito y se encuentre dentro del rango permitido para el sensor. También se implementará una versión propia del método enviar_telemetria() para mostrar el valor actual de la variable supervisada.

En la simulación se crearán dos objetos utilizando esta clase. El primero representará un sensor de temperatura y el segundo uno de iluminación. Aunque ambos pertenecerán a la misma clase, tendrán diferentes identificadores, ubicaciones, lecturas y límites de alerta.

Evidencia 3. Insertar una captura del código de la clase NodoSensor.

Programación de la clase NodoActuador

En la siguiente etapa se implementará la clase NodoActuador, que representará un dispositivo capaz de realizar una acción.

Para esta práctica se utilizará como ejemplo un ventilador. Esta clase heredará los atributos generales de NodoIoT e incorporará dos atributos privados: __estado_activo y __potencia_pwm. El atributo __estado_activo almacenará un valor booleano para indicar si el ventilador se encuentra encendido o apagado.

Por otra parte, __potencia_pwm almacenará el nivel de potencia configurado. Se desarrollará el método encender(), que permitirá activar el ventilador y asignarle un valor PWM válido. También se implementará el método apagar(), encargado de cambiar el estado del ventilador a inactivo y establecer su potencia en cero. Al igual que en la clase del sensor, se sobrescribirá el método enviar_telemetria() para presentar información específica del actuador.

Evidencia 4. Insertar una captura del código correspondiente a la clase NodoActuador.

Creación de los objetos de la red IoT

Después de implementar las clases, se crearán tres objetos para representar los dispositivos del sistema.

El primero será sensor_temp, correspondiente a un sensor de temperatura ubicado en el laboratorio, el segundo será actuador_ventilador, encargado de representar el funcionamiento de un ventilador, el tercero será sensor_luz, que simulará un sensor de iluminación ubicado en el patio cada objeto tendrá su propia identificación y dirección IP para diferenciarlo de los demás. Los tres dispositivos serán agrupados en una lista llamada red_iot.

Esta lista permitirá administrar diferentes objetos mediante una sola estructura de datos.

Implementación del bucle polimórfico

Una vez creada la lista de dispositivos, se utilizará un ciclo for para recorrer todos sus elementos, durante cada iteración, el programa ejecutará el método enviar_telemetria() sobre el objeto correspondiente, esta operación demostrará el concepto de polimorfismo,

debido a que los sensores y el actuador ejecutarán versiones diferentes del mismo método.

Cuando el ciclo llegue al sensor de temperatura, se mostrará su lectura. Cuando llegue al ventilador, se presentará su estado y nivel de potencia. Finalmente, el sensor de iluminación mostrará la información de su variable correspondiente, el beneficio de esta estructura es que permite agregar más dispositivos en el futuro sin modificar la instrucción utilizada para recorrer la red.

Implementación del menú interactivo

Para facilitar la interacción con la simulación se desarrollará un menú de opciones utilizando un ciclo while, el menú permitirá ejecutar las funciones del sistema desde la consola.

La primera opción se utilizará para conectar los dispositivos, la segunda permitirá consultar la telemetría de todos los nodos mediante el ciclo polimórfico, la tercera opción servirá para encender el ventilador y establecer un valor de potencia PWM. La cuarta permitirá apagarlo, la quinta opción se utilizará para modificar el límite de alerta del sensor de temperatura, aplicando las validaciones establecidas en el método setter; finalmente, la sexta opción permitirá salir del programa de manera controlada.

Para evitar errores ante entradas incorrectas, se utilizarán estructuras condicionales y manejo de excepciones cuando sea necesario.

Evidencia 5. Insertar una captura del menú interactivo mostrado en la consola.

6 - Resultados esperados

Al completar la implementación se espera obtener una simulación funcional compuesta por tres dispositivos IoT, los sensores deberán permitir consultar sus lecturas y mantener protegidos sus límites de alerta.

Por otra parte, el ventilador deberá responder a las instrucciones de encendido y apagado, así como almacenar correctamente su configuración de potencia.

También se espera que el sistema permita recorrer los tres objetos utilizando un único ciclo for, sin necesidad de crear instrucciones independientes para cada clase.

La implementación de la lista de dispositivos deberá demostrar la aplicación del polimorfismo, mientras que los atributos privados y los métodos de validación permitirán comprobar el encapsulamiento.

Tabla 1. Pruebas propuestas para evaluar la simulación

Prueba	Acción realizada	Resultado esperado
Conexión	Conectar los tres nodos	Todos quedan conectados
Telemetría	Consultar la red	Cada dispositivo muestra sus datos
Encendido	Activar ventilador con PWM 150	Ventilador activo

Apagado	Apagar ventilador	Estado inactivo y PWM 0
Límite válido	Asignar temperatura de alerta de 35 °C	Se acepta el valor
Límite inválido	Ingresar un valor fuera del rango configurado	Se rechaza el valor
Polimorfismo	Ejecutar el ciclo for	Cada objeto ejecuta su método correspondiente

Los resultados reales deberán registrarse después de ejecutar las pruebas, indicando si el comportamiento observado coincide con el esperado.

7 - Conclusión preliminar

El desarrollo de esta práctica permite comprender la importancia de la programación orientada a objetos para organizar sistemas integrados por diferentes dispositivos.

Mediante la creación de la clase padre NodoIoT y las clases derivadas NodoSensor y NodoActuador, se plantea una estructura que permite reutilizar atributos y métodos, evitando repetir instrucciones innecesarias, asimismo, la implementación de atributos privados y métodos de validación permite comprender cómo el encapsulamiento contribuye a mantener un mayor control sobre la información interna de los objetos.

El uso del método enviar_telemetria() en diferentes clases permite demostrar el principio de polimorfismo, ya que cada dispositivo puede responder de acuerdo con su función sin modificar la instrucción utilizada para ejecutarlo, otro aspecto importante es la implementación del menú interactivo, debido a que facilita la comunicación entre el usuario y la simulación, permitiendo ejecutar acciones y consultar el comportamiento del sistema de manera organizada.

En general, la práctica relaciona los fundamentos de programación orientada a objetos con una aplicación cercana a los sistemas IoT. Aunque se trata de una simulación, los conocimientos adquiridos pueden utilizarse como base para desarrollar proyectos que integren sensores, actuadores, microcontroladores y sistemas de supervisión.